

Reviewer Pack — SOS Degree Lower Bound via Eigenvalue Count in Moment Matrix

Entry 929f2e4b21a8 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 10:30:44 UTC

SOS Degree Lower Bound via Eigenvalue Count in Moment Matrix

Entry ID: 929f2e4b21a8 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 10:30:44 UTC

1. Conjecture

Field A (mathematical branch): Random Matrix Theory **Field B** (complexity object): SOS Degree for Max-CUT

Statement:

For a random Max-CUT instance with n variables, the minimal SOS degree required to refute it is at least $\log_2(k)$, where k is the number of non-zero eigenvalues of the corresponding moment matrix.

Rationale (proposer's reasoning):

The structure of the moment matrix's eigenvalues reflects the complexity of the polynomial constraints, and thus the SOS degree needed to capture these constraints is logarithmic in the number of non-zero eigenvalues.

Taxonomy category: SOS_DEGREE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d75b925d9f413f4a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from typing import List, Dict

def gram_matrix(instance: List[List[int]]) -> List[List[float]]:
    n = len(instance)
    G = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(i, n):
            sum_val = 0
            for k in range(n):
                sum_val += instance[i][k] * instance[j][k]
            G[i][j] = sum_val
            G[j][i] = sum_val
    return G

def eigenvalues(matrix: List[List[float]]) -> List[float]:
    n = len(matrix)
    if n == 1:
        return [matrix[0][0]]

    # Gaussian elimination to reduce the matrix
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        factor = 1 / matrix[i][i]
        for j in range(n):
            matrix[i][j] *= factor

        for j in range(n):
            if i != j:
                factor = -matrix[j][i]
                for k in range(n):
                    matrix[j][k] += factor * matrix[i][k]

    # Extract eigenvalues from the diagonal
```

```

    return [matrix[i][i] for i in range(n)]

def run_trial(seed: int) -> Dict[str, any]:
    random.seed(seed)
    n = random.randint(5, 40)
    instance = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    G = gram_matrix(instance)
    eigenvals = eigenvalues(G)
    k = sum(1 for val in eigenvals if abs(val) > 1e-6)

    sos_degree = math.ceil(math.log2(k))
    conjecture_holds = True
    counterexample = ""

    # Check if the SOS degree is at least  $\log_2(k)$ 
    if sos_degree < math.log2(k):
        conjecture_holds = False
        counterexample = f"SOS degree {sos_degree} is less than  $\log_2(k)$ "

    return {
        "metric_name": "SOS Degree",
        "metric_value": sos_degree,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_value\": {result['metric_value']}, \"instances_tested\": {result['instances_tested']}, \"conjecture_holds\": {result['conjecture_holds']}, \"counterexample\": \"{result['counterexample']}\"}}")
        results.append(result)

    mean_metric = sum(r["metric_value"] for r in results) / len(results)
    std_metric = math.sqrt(sum((r["metric_value"] - mean_metric)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric} std={std_metric} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        counterexample_desc = results[seeds.index(first_failing_seed)]["counterexample"]
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample_desc}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ture_holds": True, "counterexample": ""}
TRIAL: {"seed": 71, "metric_name": "SOS Degree", "metric_value": 5, "instances_tested": 1, "conjecture": True}
TRIAL: {"seed": 89, "metric_name": "SOS Degree", "metric_value": 4, "instances_tested": 1, "conjecture": True}
TRIAL: {"seed": 103, "metric_name": "SOS Degree", "metric_value": 6, "instances_tested": 1, "conjecture": True}
TRIAL: {"seed": 127, "metric_name": "SOS Degree", "metric_value": 3, "instances_tested": 1, "conjecture": True}
TRIAL: {"seed": 149, "metric_name": "SOS Degree", "metric_value": 4, "instances_tested": 1, "conjecture": True}
TRIAL: {"seed": 167, "metric_name": "SOS Degree", "metric_value": 5, "instances_tested": 1, "conjecture": True}
TRIAL: {"seed": 191, "metric_name": "SOS Degree", "metric_value": 6, "instances_tested": 1, "conjecture": True}
TRIAL: {"seed": 211, "metric_name": "SOS Degree", "metric_value": 5, "instances_tested": 1, "conjecture": True}
TRIAL: {"seed": 233, "metric_name": "SOS Degree", "metric_value": 4, "instances_tested": 1, "conjecture": True}
TRIAL: {"seed": 257, "metric_name": "SOS Degree", "metric_value": 5, "instances_tested": 1, "conjecture": True}

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc92df40.py", line 88, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc92df40.py", line 62, in run_trial
    eigenvals = eigenvalues(G)
                  ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc92df40.py", line 43, in eigenvalues
    factor = 1 / matrix[i][i]
              ~^~~~~~
ZeroDivisionError: float division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing full validation of the conjecture's support fraction. | next: Re-run tests with fixed division-by-zero handling and verify support_fraction >= 0.8

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109835
2	propose	ollama_remote	qwen3:8b	0	0	109026
3	preregistration	ollama_remote	qwen3:8b	0	0	24152
4	novelty	ollama_remote	qwen3:8b	0	0	20631
5	novelty	ollama_remote	qwen3:8b	0	0	13885
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15206
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10556
8	judge	ollama_remote	qwen3:8b	0	0	17666

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
---	-------	----------	-------	-----------	-----	--------------

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 320957 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/929f2e4b21a8.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/929f2e4b21a8.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/929f2e4b21a8.tar.gz (if generated)